T Swap Audit Report

Version 1.0

raihanmd.xyz

March 29, 2026

T Swap Audit Report

Raihanmd

March 29, 2026

Prepared by: Raihanmd Lead Security Researcher:

- raihanmd

Table of Contents

- Table of Contents
- Protocol Summary
  - TSwap Pools
  - Liquidity Providers
    * Why would I want to add tokens to the pool?
    * LP Example
  - Core Invariant
  - Make a swap
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings

- High
- [H-1] Missing Deadline Validation on Deposit Function
- [H-2] Incorrect Fee Calculation in getInputAmountBasedOnOutput
- [H-3] Protocol Invariant Violation: Extra Token Rewards Break Pool Equilibrium
- Medium
- [M-1] Missing Maximum Input Amount in swapExactOutput
- Low
- [L-1] Loose Deadline Check Using < Instead of <=
- [L-2] Incorrect LiquidityAdded Event Parameter Order
- Informational
- [I-1] Unused Error Definition
- [I-2] Missing Indexed Parameters in PoolCreated Event
- [I-3] Magic Numbers in Fee Calculations
- [I-4] Missing Zero Address Validation in Constructor

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: Uniswap Explained

TSwap Pools

The protocol starts as simply a `PoolFactory` contract. This contract is used to create new "pools" of tokens. It helps make sure every pool token uses the correct logic. But all the magic is in each `TSwapPool` contract.

You can think of each `TSwapPool` contract as it's own exchange between exactly 2 assets. Any ERC20 and the WETH token. These pools allow users to permissionlessly swap between an ERC20 that has a pool and WETH. Once enough pools are created, users can easily "hop" between supported ERC20s.

For example:

1. User A has 10 USDC
2. They want to use it to buy DAI
3. They `swap` their 10 USDC -> WETH in the USDC/WETH pool

4. Then they `swap` their WETH -> DAI in the DAI/WETH pool

Every pool is a pair of `TOKEN X` & `WETH`.

There are 2 functions users can call to swap tokens in the pool.

- `swapExactInput`
- `swapExactOutput`

We will talk about what those do in a little.

Liquidity Providers

In order for the system to work, users have to provide liquidity, aka, "add tokens into the pool".

Why would I want to add tokens to the pool?

The TSwap protocol accrues fees from users who make swaps. Every swap has a `0.3` fee, represented in `getInputAmountBasedOnOutput` and `getOutputAmountBasedOnInput`. Each applies a `997` out of `1000` multiplier. That fee stays in the protocol.

When you deposit tokens into the protocol, you are rewarded with an LP token. You'll notice `TSwapPool` inherits the `ERC20` contract. This is because the `TSwapPool` gives out an ERC20 when Liquidity Providers (LP)s deposit tokens. This represents their share of the pool, how much they put in. When users swap funds, 0.03% of the swap stays in the pool, netting LPs a small profit.

LP Example

1. LP A adds 1,000 WETH & 1,000 USDC to the USDC/WETH pool
    1. They gain 1,000 LP tokens
2. LP B adds 500 WETH & 500 USDC to the USDC/WETH pool
    1. They gain 500 LP tokens
3. There are now 1,500 WETH & 1,500 USDC in the pool
4. User A swaps 100 USDC -> 100 WETH.
    1. The pool takes 0.3%, aka 0.3 USDC.
    2. The pool balance is now 1,400.3 WETH & 1,600 USDC
    3. aka: They send the pool 100 USDC, and the pool sends them 99.7 WETH

Note, in practice, the pool would have slightly different values than 1,400.3 WETH & 1,600 USDC due to the math below.

Core Invariant

Our system works because the ratio of Token A & WETH will always stay the same. Well, for the most part. Since we add fees, our invariant technically increases.

The fundamental equation: $x \times y = k$

Where:

- x = Token Balance X
- y = Token Balance Y
- k = The constant product (invariant)

Without Fees:

$$x \times y = (x + \Delta x) \times (y - \Delta y)$$

Where:

- Δx = Change of token balance X
- Δy = Change of token balance Y
- $\beta = \Delta y / y$
- $\alpha = \Delta x / x$

The invariant equations are:

$$\Delta x = \frac{\beta}{1 - \beta} \times x$$

$$\Delta y = \frac{\alpha}{1 + \alpha} \times y$$

With 0.3% Fee:

- ρ = fee (between 0 & 1)
- $\gamma = 1 - \rho$ (fee multiplier)

The adjusted invariant equations become:

$$\Delta x = \frac{\beta}{1-\beta} \times \frac{1}{\gamma} \times x$$

$$\Delta y = \frac{\alpha\gamma}{1+\alpha\gamma} \times y$$

Our protocol should always follow this invariant in order to keep swapping correctly!

Make a swap

After a pool has liquidity, there are 2 functions users can call to swap tokens in the pool.

- `swapExactInput`
- `swapExactOutput`

A user can either choose exactly how much to input (ie: I want to use 10 USDC to get however much WETH the market says it is), or they can choose exactly how much they want to get out (ie: I want to get 10 WETH from however much USDC the market says it is.)

This codebase is based loosely on Uniswap v1

Disclaimer

The raihanmd team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 7bd52c6a75f1115b23e0cff0fa16f7c522703fcd

Scope

- In Scope:

```
./src/
#-- PoolFactory.sol
#-- TSwapPool.sol
```

- Solc Version: 0.8.20

Roles

There are 3 types of actors in the T-Swap protocol:

Actors:

1. Protocol Owner

- Deploy the PoolFactory and TSwapPool contracts
- Set parameters (WETH token address, fee structure)
- Create new trading pools via PoolFactory

2. Liquidity Providers (LPs)

- Deposit ERC20 tokens and WETH into pools to provide liquidity
- Receive LP tokens representing their share of the pool
- Earn 0.3% fee revenue from swaps
- Withdraw liquidity and recover LP tokens at any time

3. Traders/Swappers

- Execute permissionless token swaps via `swapExactInput()` or `swapExactOutput()`
- Trade any ERC20 that has a pool against WETH
- Pay 0.3% fee on all swaps
- No special permissions required

Executive Summary

We spent 4 hour with 1 security researcher, we are using tools foundry as a framework for testing

Issues found

Severity	Number of issues found
HIGH	3
MEDIUM	1
LOW	3
INFO	3
TOTAL	11

Findings

High

[H-1] Missing Deadline Validation on Deposit Function

Description:

The `TSwapPool::deposit()` function is missing the `revertIfDeadlinePassed(deadline)` modifier on its signature. While the parameter is accepted, the validation is not performed. This allows liquidity providers to submit transactions with past deadlines that still get executed, violating the user's explicit time constraints.

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline // @>
)
    external
    revertIfZero(wethToDeposit)
    // Missing: modifier revertIfDeadlinePassed(deadline)
    returns (uint256 liquidityTokensToMint)
{
    // ... rest of function
```

```
}
```

Risk:

This is a high-severity issue because deadlines are a critical protection mechanism in AMM protocols. Users set deadlines to protect against unexpected market movements and MEV attacks. When a deadline is ignored, the user's slippage protection is completely bypassed.

Impact: HIGH

- LP transactions can be delayed and executed at unfavorable rates
- Long-pending transactions can execute at stale prices, causing LP loss
- Front-runners and MEV bots can exploit the missing deadline check
- Users lose protection against sandwich attacks and price manipulation
- Protocol appears to respect deadlines but silently ignores them

Likelihood: HIGH

- High - Any MEV searcher or malicious validator can exploit this
- Likely to occur in volatile market conditions where price moves significantly
- Mempool monitoring tools can identify transactions with outdated deadlines

Proof of Concept:

```
function test__h1_missing_deadline_check_on_deposit() public {
    uint256 wethToDeposit = 10 ether;
    uint256 poolTokenAmount = 100 ether;

    weth.mint(lp, wethToDeposit);
    poolToken.mint(lp, poolTokenAmount + 1 ether);

    vm.startPrank(lp);
    weth.approve(address(pool), type(uint256).max);
    poolToken.approve(address(pool), type(uint256).max);

    // Set deadline to block.timestamp - 1 (in the past)
    uint64 pastDeadline = uint64(block.timestamp - 1);

    uint256 expectedPoolTokens = pool.getPoolTokensToDepositBasedOnWeth
        (wethToDeposit);

    // This call should fail but passes because revertIfDeadlinePassed
        is missing
    uint256 liquidityTokensMinted =
        pool.deposit(wethToDeposit, 0, expectedPoolTokens, pastDeadline
            );

    // Verify deposit went through despite past deadline
```

```
    assertGt(liquidityTokensMinted, 0, "Deposit should have reverted due to past deadline");

    vm.stopPrank();
}
```

Recommended Mitigation:

Add the `revertIfDeadlinePassed` modifier to the `deposit()` function signature:

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
+   revertIfDeadlinePassed(deadline)
    revertIfZero(wethToDeposit)
    returns (uint256 liquidityTokensToMint)
```

[H-2] Incorrect Fee Calculation in getInputAmountBasedOnOutput

Description:

The `TSwapPool::getInputAmountBasedOnOutput()` function uses an incorrect divisor (10_000 instead of 1_000) in its fee calculation, effectively charging more than 0.3% fee. This causes users to pay significantly more tokens than they should for exact output swaps.

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
)
    public
    pure
    revertIfZero(outputAmount)
    revertIfZero(outputReserves)
    returns (uint256 inputAmount)
{
    return ((inputReserves * outputAmount) * 10000) / ((outputReserves - outputAmount) * 997); // @>
}
```

The correct formula should use 1_000 (0.3% fee = 997/1000):

Risk:

This is a critical fee calculation bug that directly extracts value from users. Every exact-output swap will charge approximately 10x more in fees than intended. This breaks the protocol's economic model and incentives.

Impact: HIGH

- Users pay 90% fee instead of 0.03% (10000 vs 1000)
- Severely reduces LP competitiveness against correctly-implemented AMMs
- Users experience unexpected high slippage and direct financial loss
- Protocol appears to have 0.03% fee but actually charges ~0.3% or more
- Revenue model is broken as platform becomes uncompetitive

Likelihood: HIGH

- Guaranteed to occur - Every swap using `TSwapPool::swapExactOutput()` is affected
- Compound impact over time as TVL may decrease due to poor pricing
- Users will switch to better-priced AMMs

Proof of Concept:

```
function test__h2_incorrect_fee_calculation() public {
    uint256 wethReserves = weth.balanceOf(address(pool));
    uint256 poolTokenReserves = poolToken.balanceOf(address(pool));
    uint256 desiredWethOut = 1 ether;

    // Get the required pool tokens using the BUGGY function
    uint256 buggyInputAmount = pool.getInputAmountBasedOnOutput(
        desiredWethOut, poolTokenReserves, wethReserves);

    // Calculate what it SHOULD be with correct 1_000 divisor
    uint256 correctInputAmount = ((poolTokenReserves * desiredWethOut)
        * 1000)
         / ((wethReserves - desiredWethOut) * 997);

    // Buggy version requires more input
    assertGt(buggyInputAmount, correctInputAmount, "Buggy function
        charges excess fee");
}
```

Recommended Mitigation:

Change the divisor from 10_000 to 1_000:

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
```

```
)
    public
    pure
    revertIfZero(outputAmount)
    revertIfZero(outputReserves)
    returns (uint256 inputAmount)
{
-   return ((inputReserves * outputAmount) * 10000) / ((outputReserves - outputAmount) * 997);
+   return ((inputReserves * outputAmount) * 1000) / ((outputReserves - outputAmount) * 997);
}
```

[H-3] Protocol Invariant Violation: Extra Token Rewards Break Pool Equilibrium

Description:

The `TSwapPool::_swap()` function rewards every 10th swap with 1 WETH bonus token, but this token is transferred directly from the pool without corresponding accounting. This breaks the fundamental x*y=k invariant and can drain pool reserves.

```
function _swap(IERC20 inputToken, uint256 inputAmount, IERC20 outputToken, uint256 outputAmount) private {
    if (_isUnknown(inputToken) || _isUnknown(outputToken) || inputToken == outputToken) {
        revert TSwapPool__InvalidToken();
    }

    swap_count++; // @>
    if (swap_count >= SWAP_COUNT_MAX) {
        swap_count = 0;
        outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
    } // @>
    emit Swap(msg.sender, inputToken, inputAmount, outputToken, outputAmount);

    inputToken.safeTransferFrom(msg.sender, address(this), inputAmount);
    outputToken.safeTransfer(msg.sender, outputAmount);
}
```

Risk:

The bonus token transfer violates the AMM invariant without proper accounting. This can lead to pool insolvency and breaks the mathematical guarantees that make AMMs secure.

Impact: HIGH

- Unaccounted token outflows break the x*y=k invariant
- Pool reserves gradually deplete due to bonus rewards
- Eventually pool becomes insolvent and cannot fulfill swaps
- Users who deposit liquidity later suffer from diluted pool reserves
- Smart contracts depending on invariant guarantees may fail

Likelihood: HIGH

- Guaranteed to occur - Every 10th swap triggers the vulnerability
- Accumulates over many swaps, leading to eventual pool insolvency
- More severe with high swap volume

Proof of Concept:

```
function test__m4_fee_on_transfer_breaks_invariant() public {
    uint256 initialWethReserves = weth.balanceOf(address(pool));
    uint256 initialPoolTokenReserves = poolToken.balanceOf(address(pool));
    uint256 initialK = initialWethReserves * initialPoolTokenReserves;

    // Perform 10 swaps to trigger bonus
    for (uint256 i = 0; i < 10; i++) {
        uint256 swapAmount = 1 ether / 100;

        if (poolToken.balanceOf(swapper) < swapAmount) {
            poolToken.mint(swapper, swapAmount);
        }

        vm.startPrank(swapper);
        poolToken.approve(address(pool), swapAmount);

        uint256 wethBefore = weth.balanceOf(swapper);
        pool.swapExactInput(poolToken, swapAmount, weth, 0, uint64(block.timestamp + 1000));
        uint256 wethAfter = weth.balanceOf(swapper);

        vm.stopPrank();
    }

    uint256 finalWethReserves = weth.balanceOf(address(pool));
    uint256 finalPoolTokenReserves = poolToken.balanceOf(address(pool));
    uint256 finalK = finalWethReserves * finalPoolTokenReserves;

    // K violates invariant due to unaccounted bonus transfer
    assertLt(finalK, initialK + 1e18, "Bonus transfer breaks invariant");
}
```

Recommended Mitigation:

Remove the bonus reward system or properly account for it within the invariant:

```
function _swap(IERC20 inputToken, uint256 inputAmount, IERC20 outputToken, uint256 outputAmount) private {
    if (_isUnknown(inputToken) || _isUnknown(outputToken) || inputToken == outputToken) {
        revert TSwapPool__InvalidToken();
    }

-   swap_count++;
-   if (swap_count >= SWAP_COUNT_MAX) {
-       swap_count = 0;
-       outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
-   }
    emit Swap(msg.sender, inputToken, inputAmount, outputToken, outputAmount);

    inputToken.safeTransferFrom(msg.sender, address(this), inputAmount);
    outputToken.safeTransfer(msg.sender, outputAmount);
}
```

Medium

[M-1] Missing Maximum Input Amount in swapExactOutput

Description:

The `TSwapPool::swapExactOutput()` function lacks a `maxInputAmount` parameter, preventing users from protecting themselves against excessive slippage or MEV attacks. Users cannot limit their maximum spend when executing exact-output swaps.

```
// ! @audit missing maximum input amount to prevent griefing attack, slippage protection
function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
    uint256 outputAmount,
    uint64 deadline
)
    public
    revertIfZero(outputAmount)
    revertIfDeadlinePassed(deadline)
    returns (uint256 inputAmount)
{
```

```
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

    inputAmount = getInputAmountBasedOnOutput(outputAmount,
        inputReserves, outputReserves);

    _swap(inputToken, inputAmount, outputToken, outputAmount);
}
```

Risk:

Without maximum input checks, users can suffer from extreme slippage, MEV extraction, or griefing attacks where the required input amount becomes unexpectedly high between transaction signing and execution.

Impact: MEDIUM

- Users can pay significantly more than expected for their desired output
- MEV bots can sandwich transactions and extract maximum slippage
- No protection against pool state changes between signing and execution
- Cascading failures in dependent protocols that expect slippage bounds

Likelihood: HIGH

- High in volatile market conditions
- Guaranteed during MEV-driven congestion periods
- Any smart contract calling this function is at risk

Proof of Concept:

```
function test__m3_missing_max_input_slippage_protection() public {
    uint256 wethReserves = weth.balanceOf(address(pool));

    uint256 largeTxOut = (wethReserves * 5) / 100; // 5% of reserves

    uint256 requiredInput =
        pool.getInputAmountBasedOnOutput(largeTxOut, poolToken.
            balanceOf(address(pool)), wethReserves);

    poolToken.mint(swapper, requiredInput * 2);

    vm.startPrank(swapper);
    poolToken.approve(address(pool), type(uint256).max);

    // No maximum input check - user cannot protect against slippage
    pool.swapExactOutput(poolToken, weth, largeTxOut, uint64(block.
        timestamp + 1000));

    vm.stopPrank();
```

```

    assertEq(weth.balanceOf(swapper), largeTxOut, "Should have received exact output");
}
```

Recommended Mitigation:

Add `maxInputAmount` parameter with validation:

```
function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
    uint256 outputAmount,
+   uint256 maxInputAmount,
    uint64 deadline
)
    public
    revertIfZero(outputAmount)
    revertIfDeadlinePassed(deadline)
    returns (uint256 inputAmount)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

    inputAmount = getInputAmountBasedOnOutput(outputAmount, inputReserves, outputReserves);

+   if (inputAmount > maxInputAmount) {
+       revert TSwapPool__InputTooHigh(inputAmount, maxInputAmount);
+   }

    _swap(inputToken, inputAmount, outputToken, outputAmount);
}
```

Low

[L-1] Loose Deadline Check Using < Instead of <=

Description:

The TSwapPool constructor does not validate that the `poolToken` and `wethToken` parameters are not `address(0)`. This allows the creation of pools with invalid token addresses, which would cause the pool to malfunction.

```
constructor(
    address poolToken,
    address wethToken,
    string memory liquidityTokenName,
```

```
    string memory liquidityTokenSymbol
)
    ERC20(liquidityTokenName, liquidityTokenSymbol)
{
    i_wethToken = IERC20(wethToken); // @>
    i_poolToken = IERC20(poolToken); // @>
}
```

Risk:

Low severity as this requires owner to make a mistake during deployment. However, zero address checks are defensive programming best practice.

Impact: LOW

- If owner accidentally passes address(0), pool would break
- Broken pools might not be easily identifiable on-chain
- Users might add liquidity to broken pools and lose funds

Likelihood: HIGH

- Low - Unlikely if deployment is tested, but possible with manual errors
- Defensive measure for protocol resilience

Proof of Concept:

```
function test__l2_missing_zero_address_check() public {
    // This test documents that there's no zero address validation
    // A pool was successfully created in setUp, showing constructor accepts parameters
    // Without validation, if owner passes address(0) by mistake, pool would break

    // Verify the pool exists and is usable (constructor doesn't check for zero addresses)
    assertNotEq(address(pool), address(0), "Pool should exist");
    assertNotEq(pool.getPoolToken(), address(0), "Pool token should not be zero");
    assertNotEq(pool.getWeth(), address(0), "WETH should not be zero");
}
```

Recommended Mitigation:

Add zero address checks in the constructor:

```
constructor(
    address poolToken,
    address wethToken,
    string memory liquidityTokenName,
    string memory liquidityTokenSymbol
```

```
)
    ERC20(liquidityTokenName, liquidityTokenSymbol)
{
+   if (poolToken == address(0) || wethToken == address(0)) {
+       revert InvalidAddress();
+   }
    i_wethToken = IERC20(wethToken);
    i_poolToken = IERC20(poolToken);
}
```

[L-2] Incorrect LiquidityAdded Event Parameter Order

Description:

The `LiquidityAdded` event emits parameters in the wrong order. The function signature shows `(wethDeposited, poolTokensDeposited)` but the implementation emits them swapped as `(poolTokensDeposited, wethDeposited)`.

```
function _addLiquidityMintAndTransfer(
    uint256 wethToDeposit,
    uint256 poolTokensToDeposit,
    uint256 liquidityTokensToMint
)
    private
{
    _mint(msg.sender, liquidityTokensToMint);
    emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit); // @>
    // ...
}
```

Risk:

Event parameter misalignment confuses off-chain indexers and analytics tools that rely on event parsing. This causes data corruption in monitoring systems and user dashboards.

Impact: LOW

- Off-chain monitoring systems interpret swapped values
- Analytics dashboards show incorrect deposit amounts
- User transaction history displays wrong amounts
- Blockchain explorers show confusing event logs

Likelihood: HIGH

- Low direct user impact but affects all off-chain infrastructure
- Becomes obvious when monitoring/analytics are implemented

Proof of Concept:

```
function test__i1_wrong_event_parameter_order() public {
    uint256 wethToDeposit = 5 ether;
    uint256 expectedPoolTokens = pool.getPoolTokensToDepositBasedOnWeth
        (wethToDeposit);

    weth.mint(lp, wethToDeposit);
    poolToken.mint(lp, expectedPoolTokens);

    vm.startPrank(lp);
    weth.approve(address(pool), type(uint256).max);
    poolToken.approve(address(pool), type(uint256).max);

    // Store balances before deposit
    uint256 wethBefore = weth.balanceOf(address(pool));
    uint256 poolTokenBefore = poolToken.balanceOf(address(pool));

    // Execute deposit
    pool.deposit(wethToDeposit, 0, expectedPoolTokens, uint64(block.
        timestamp + 1000));

    // Verify balances changed correctly
    uint256 wethAfter = weth.balanceOf(address(pool));
    uint256 poolTokenAfter = poolToken.balanceOf(address(pool));

    assertEq(wethAfter - wethBefore, wethToDeposit, "WETH should be
        deposited");
    assertEq(poolTokenAfter - poolTokenBefore, expectedPoolTokens, "
        Pool tokens should be deposited");

    vm.stopPrank();
}
```

Recommended Mitigation:

Fix the event parameter order to match the function intent:

```
function _addLiquidityMintAndTransfer(
    uint256 wethToDeposit,
    uint256 poolTokensToDeposit,
    uint256 liquidityTokensToMint
)
    private
{
    _mint(msg.sender, liquidityTokensToMint);
-   emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit)
    ;
```

```
+    emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);

     // Interactions
     i_wethToken.safeTransferFrom(msg.sender, address(this), wethToDeposit);
     i_poolToken.safeTransferFrom(msg.sender, address(this), poolTokensToDeposit);
}
```

Informational

[I-1] Unused Error Definition

Description:

The `PoolFactory__PoolDoesNotExist` error is defined but never used in the contract:

```
error PoolFactory__PoolAlreadyExists(address tokenAddress);
error PoolFactory__PoolDoesNotExist(address tokenAddress); // @>
```

Recommended Mitigation:

Remove the unused error or implement its usage:

```
error PoolFactory__PoolAlreadyExists(address tokenAddress);
-error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Missing Indexed Parameters in PoolCreated Event

Description:

The `PoolCreated` event does not mark parameters as `indexed`, making it harder to filter events by pool or token address:

```
// ! @audit-1 indexed should be better
event PoolCreated(address tokenAddress, address poolAddress);
```

Recommended Mitigation:

Add `indexed` keyword to frequently filtered parameters:

```
-event PoolCreated(address tokenAddress, address poolAddress);
+event PoolCreated(address indexed tokenAddress, address indexed
    poolAddress);
```

[I-3] Magic Numbers in Fee Calculations

Description:

The fee calculations use hardcoded magic numbers (997, 1000) without explanation:

```
// ! @audit-4 magic numbers
uint256 inputAmountMinusFee = inputAmount * 997;
uint256 denominator = (inputReserves * 1000) + inputAmountMinusFee;
```

Recommended Mitigation:

Define constants for clarity:

```
+uint256 private constant FEE_PERCENTAGE = 997; // 0.3% fee = 997/1000
+uint256 private constant FEE_DENOMINATOR = 1000;

function getOutputAmountBasedOnInput(...) public pure returns (uint256
    outputAmount) {
-    uint256 inputAmountMinusFee = inputAmount * 997;
+    uint256 inputAmountMinusFee = inputAmount * FEE_PERCENTAGE;
-    uint256 denominator = (inputReserves * 1000) + inputAmountMinusFee;
+    uint256 denominator = (inputReserves * FEE_DENOMINATOR) +
    inputAmountMinusFee;
     return numerator / denominator;
}
```

[I-4] Missing Zero Address Validation in Constructor

Description:

The TSwapPool constructor does not validate that the `poolToken` and `wethToken` parameters are not `address(0)`. This is a defensive programming best practice. Only occurs through owner negligence during deployment, making it INFORMATIONAL rather than LOW severity.

```
constructor(
    address poolToken,
    address wethToken,
    string memory liquidityTokenName,
```

```
    string memory liquidityTokenSymbol
)
    ERC20(liquidityTokenName, liquidityTokenSymbol)
{
    i_wethToken = IERC20(wethToken); // @> no zero address check
    i_poolToken = IERC20(poolToken); // @> no zero address check
}
```

Recommended Mitigation:

Add zero address checks in the constructor:

```
constructor(
    address poolToken,
    address wethToken,
    string memory liquidityTokenName,
    string memory liquidityTokenSymbol
)
    ERC20(liquidityTokenName, liquidityTokenSymbol)
{
+   if (poolToken == address(0) || wethToken == address(0)) {
+       revert InvalidAddress();
+   }
    i_wethToken = IERC20(wethToken);
    i_poolToken = IERC20(poolToken);
}
```